A MINI PROJECT REPORT

on

Diabetes Classification & Prediction

As a part of

Data Science (CE0630)

Submitted by

Yug Patel

(IU2341230117)

In fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

INSTITUTE OF TECHNOLOGY AND ENGINEERING

INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ

AHMEDABAD-382115, GUJARAT, INDIA

WEB: www.indusuni.ac.in

APRIL 2026

Problem Statement : Diabetes Classification & Prediction

Data Set: Diabetes Dataset

(https://www.kaggle.com/datasets/imtkaggleteam/diabetes)

1) Upload Dataset :

```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files diabetes.csv
diabetes.csv(text/csv) - 37065 bytes, last modified: 4/23/2026 - 100% done
Saving diabetes.csv to diabetes.csv
```

2) Import Libraries

```
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

3) Load Dataset :

```
df = pd.read_csv('diabetes.csv')
df.head()
```

	id	chol	stab.glu	hdl	ratio	glyhb	location	age	gender	height	weight	frame	bp.1s
0	1000	203.0	82	56.0	3.6	4.31	Buckingham	46	female	62.0	121.0	medium	118.0
1	1001	165.0	97	24.0	6.9	4.44	Buckingham	29	female	64.0	218.0	large	112.0
2	1002	228.0	92	37.0	6.2	4.64	Buckingham	58	female	61.0	256.0	large	190.0
3	1003	78.0	93	12.0	6.5	4.63	Buckingham	67	male	67.0	119.0	large	110.0
4	1005	249.0	90	28.0	8.9	7.72	Buckingham	64	male	68.0	183.0	medium	138.0

4) Data Cleaning :

```
# Replace 0 → NaN in selected columns
cols = ['stab.glu','chol','hdl','ratio','glyhb',

'bp.1s','bp.1d','bp.2s','bp.2d','waist','hip']

df[cols] = df[cols].replace(0, np.nan)

# Fill missing values
df.fillna({
      col: df[col].mean() for col in df.select_dtypes(include=np.number).columns
}, inplace=True)

df.fillna({
     col: df[col].mode()[0] for col in df.select_dtypes(exclude=np.number).columns
}, inplace=True)

# Create Outcome
df['Outcome'] = (df['glyhb'] >= 6.5).astype(int)

# Encode categorical columns
from sklearn.preprocessing import LabelEncoder

for col in ['gender', 'location', 'frame']:
   if col in df.columns:
                              df[col] =
```

```
LabelEncoder().fit_transform(df[col]
)

# First 5 rows for checking after clean up
print("After Clean Up: \n",df.head())
```

```
After Clean Up:
     id   chol  stab.glu   hdl  ratio  glyhb  location  age  gender  height  \
0  1000  203.0        82  56.0    3.6   4.31         0   46       0    62.0
1  1001  165.0        97  24.0    6.9   4.44         0   29       0    64.0
2  1002  228.0        92  37.0    6.2   4.64         0   58       0    61.0
3  1003   78.0        93  12.0    6.5   4.63         0   67       1    67.0
4  1005  249.0        90  28.0    8.9   7.72         0   64       1    68.0

   weight  frame  bp.1s  bp.1d       bp.2s      bp.2d  waist   hip  time.ppn  \
0   121.0      1  118.0   59.0  152.382979  92.524823   29.0  38.0     720.0
1   218.0      0  112.0   68.0  152.382979  92.524823   46.0  48.0     360.0
2   256.0      0  190.0   92.0  185.000000  92.000000   49.0  57.0     180.0
3   119.0      0  110.0   50.0  152.382979  92.524823   33.0  38.0     480.0
4   183.0      1  138.0   80.0  152.382979  92.524823   44.0  41.0     300.0

   Outcome
0        0
1        0
2        0
3        0
4        1
```

5) Statistical Analysis:

```
from scipy import stats
import numpy as np

col = "stab.glu"

mean_val = df[col].mean()
median_val = df[col].median()
mode_val = df[col].mode()[0]
skew_val = stats.skew(df[col])
kurt_val = stats.kurtosis(df[col])

print(f"--- Statistical Analysis ({col}) ---")
print("Mean:", mean_val)
print("Median:", median_val)
print("Mode:", mode_val)
```

```
print("Skew:", skew_val)
print("Kurtosis:", kurt_val)
```

```
--- Statistical Analysis (stab.glu) ---
Mean: 106.67245657568238
Median: 89.0
Mode: 85
Skew: 2.755824376240672
Kurtosis: 8.158451373717767
```

6) Correlation Analysis :

```
corr = df.corr(numeric_only=True)['Outcome'].sort_values(ascending=False)
print(corr)
```

```
Outcome     1.000000
glyhb       0.855214
stab.glu    0.685887
age         0.311686
ratio       0.283366
waist       0.220417
bp.1s       0.219457
chol        0.207415
weight      0.149066
hip         0.135763
bp.2s       0.051327
bp.1d       0.046567
time.ppn    0.029982
gender      0.023816
height      0.013381
id          0.008627
location   -0.010011
bp.2d      -0.079564
hdl        -0.126135
frame      -0.161549
Name: Outcome, dtype: float64
```

7) Hypothesis Testing :

```
population_mean = 0.5
std_dev = df['Outcome'].std()
n = len(df)

z = (mean_val - population_mean) / (std_dev / np.sqrt(n))
print("Z value:", z)
```

```
Z value: 5787.818587864607
```

8) Feature Engineering:

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Features and target
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

# Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (322, 19)
Test shape: (81, 19)
```

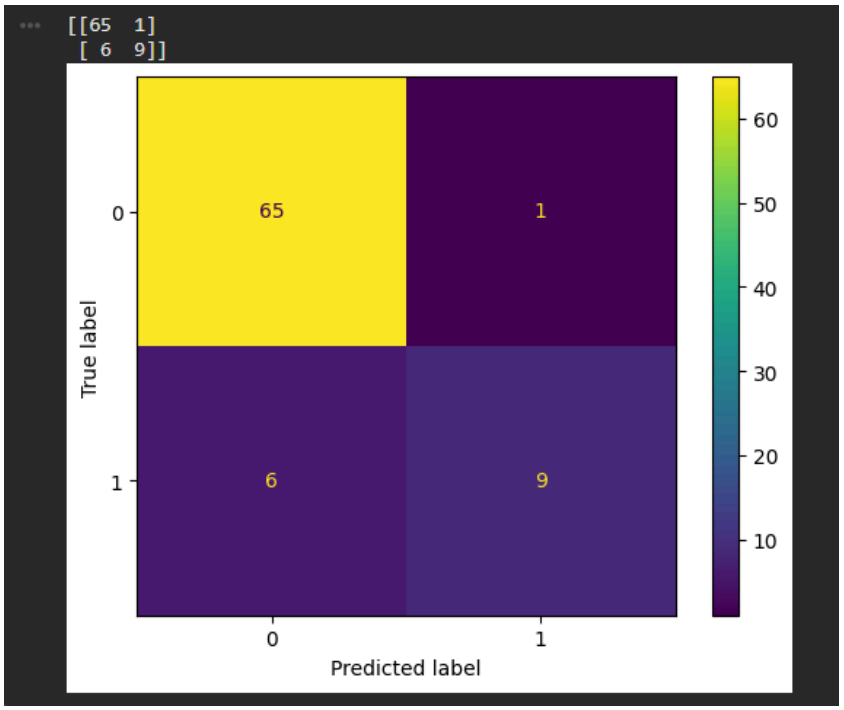
9) Logistic Model :

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(y_pred)
```

```
[0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 1 0 0]
```

10) Confusion Matrix:

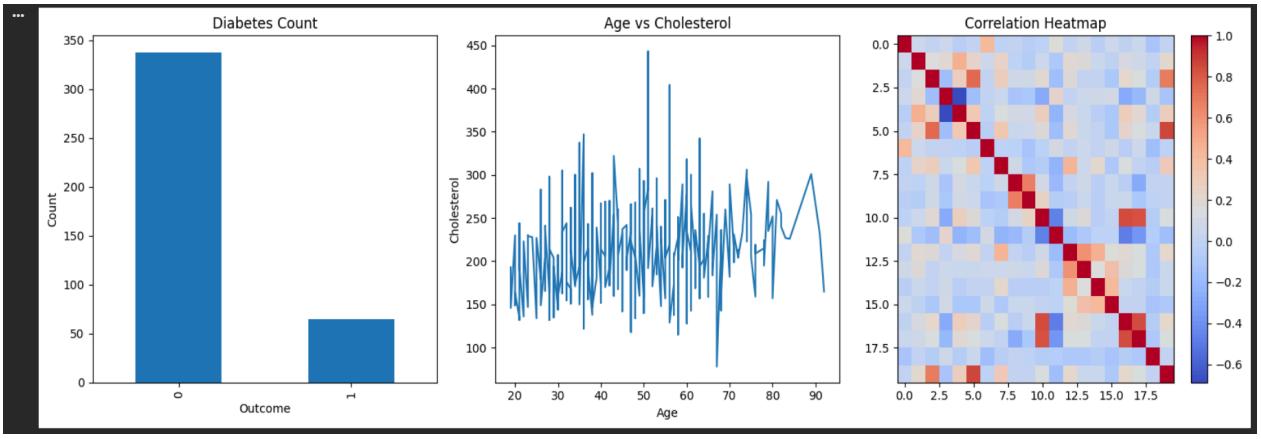
```
cm = confusion_matrix(y_test, y_pred)
print(cm)

ConfusionMatrixDisplay(cm).plot()
plt.show()
```



11) Graphs :

```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(15,5))

# 1. Bar Plot (Outcome Count)
plt.subplot(1,3,1)
df["Outcome"].value_counts().plot(kind='bar')
plt.title("Diabetes Count")
plt.xlabel("Outcome")
```

```
plt.ylabel("Count")

# 2. Line Plot (Age vs Cholesterol Trend)
plt.subplot(1,3,2)
df_sorted = df.sort_values(by="age")
plt.plot(df_sorted["age"], df_sorted["chol"])
plt.title("Age vs Cholesterol")
plt.xlabel("Age")
plt.ylabel("Cholesterol")

# 3. Correlation Heatmap
plt.subplot(1,3,3)
corr_matrix = df.corr(numeric_only=True)
plt.imshow(corr_matrix, cmap="coolwarm", aspect="auto")
plt.title("Correlation Heatmap")
plt.colorbar()

plt.tight_layout()
plt.show()
```



Conclusion: The diabetes dataset was successfully preprocessed by handling missing values, replacing invalid entries, and encoding categorical variables.Statistical analysis and correlation study helped in understanding the distribution of medical attributes and identifying important features influencing diabetes, such as glucose level, cholesterol,etc.

A Logistic Regression model was implemented for classification, and the dataset was split into training and testing sets after scaling the features. The model performance was evaluated using a confusion matrix and accuracy metrics, showing that the model is capable of distinguishing between diabetic and non-diabetic patients with reasonable effectiveness.

Various visualizations, including bar plot, line plot, and correlation heatmap, were used to explore patterns and relationships in the data, making the analysis more interpretable.

Overall, the project demonstrates how data preprocessing, statistical analysis, and machine learning techniques can be combined to build a predictive model for diabetes classification. While the model provides satisfactory results, further improvements can be achieved using advanced algorithms and feature engineering techniques.